9/28/2026

BUSINESS BUZZ - IMPLEMENTATION DOCUMENT

BY Buzz Tech solutions

Group Members

Mnoneleli Nomboyo – 223444731
Anathi Silangwe – 227565649
Talita Paca - 228141400

Table of Contents

Introduction 3
1.1 Choice of Tools 3
1.1.1 Visual Studio 2022 3
1.1.2 Visual Studio Code 4
1.1.3 ASP.NET Core 4
1.1.4 C# 4
1.1.5 Microsoft SQL Server 5
1.1.6 Entity Framework Core 5
1.1.7 HTML5 5
1.1.8 CSS3 6
1.1.9 Bootstrap 5.3 6
1.1.10 JavaScript 6
1.1.11 Git and GitHub 7
1.2 Extracts of Complex Code 7
1.2.1 Authentication and Password Hashing 7
1.2.2 Business Profile Creation and Validation 8
1.2.3 Personalized Journey Generation 9
1.3 Source Code References 10
1.3.1 Password Hashing and Verification 10
1.3.2 ASP.NET Core Session State 11
1.3.3 Entity Framework Core Database Migrations 11
1.3.4 Entity Framework Core Object-Relational Mapping 12
1.3.5 Bootstrap 5.3 Frontend Toolkit 12
1.4 Problems Encountered 13
1.4.1 Entity Framework Core Migration and Database Configuration 13
1.4.2 Authentication Service Registration 13
1.4.3 SQL Server Connection Timeout During Login 14
1.4.4 Updating the Database as the Application Evolved 14
1.4.5 Configuring the ASP.NET Core Frontend Project 14

1.4.6 Static Files and Web Root Path 14
1.4.7 Navigation Between Frontend Pages 15
1.4.8 Developing the Frontend and Backend in Parallel 15
1.4.9 Collaborating Through Git and GitHub 16
References 17

IMPLEMENTATION

Introduction

This document describes the implementation of **Business Buzz**, a centralized web-based business registration and compliance management system developed as an academic prototype. The purpose of Business Buzz is to provide entrepreneurs with a single environment in which they can prepare business information, follow simulated registration and compliance processes, generate relevant business documents, and monitor their progress towards business readiness.

The implementation of Business Buzz consists of a web-based frontend, an ASP.NET Core backend, and a Microsoft SQL Server database. The system supports three primary user roles: **Entrepreneur, Approver, and Bank**. Each role is provided with functionality relevant to its responsibilities within the system. The Entrepreneur follows the business registration and compliance journey, the Approver reviews relevant applications and information, and the Bank reviews relevant simulated registration and compliance information when conducting a simulated financial-service assessment.

Business Buzz models business processes associated with entities such as the Companies and Intellectual Property Commission (CIPC), South African Revenue Service (SARS), Unemployment Insurance Fund (UIF), and Home Affairs. These processes are simulated within the academic prototype and do not represent direct integration with official institutional systems.

This document discusses the tools and technologies selected for the implementation of Business Buzz, extracts of significant code developed for the system, external framework and implementation resources used during development, and challenges encountered by the development team.

1.1 Choice of Tools

The Business Buzz application was developed using a combination of development environments, backend technologies, database technologies, frontend technologies, frameworks, and version-control tools. The tools were selected according to the technical requirements of the system and the need for different members of the development team to work on different components of the application in parallel.

1.1.1 Visual Studio 2022

Visual Studio 2022 was used as one of the primary integrated development environments (IDEs) for the Business Buzz project, particularly for frontend development and management of the ASP.NET Core project structure.

Visual Studio 2022 was selected because it provides integrated support for developing .NET applications, including source-code editing, building, debugging, project management, and

source-control integration. These capabilities assisted the team in identifying implementation problems and testing the application locally.

According to Microsoft (n.d.-a), Visual Studio provides an integrated development environment containing tools for editing, building, debugging, and managing software projects.

1.1.2 Visual Studio Code

Visual Studio Code was also used during the implementation of Business Buzz, particularly for backend development.

Visual Studio Code was selected because it provides a lightweight development environment while supporting C#, .NET, Git, terminal commands, debugging, and extensions.

The use of Visual Studio Code allowed backend development to take place within a preferred development environment while still contributing to the same Business Buzz GitHub repository. This was useful because the frontend and backend were developed by different team members in parallel.

Microsoft (n.d.-b) describes Visual Studio Code as a source-code editor that supports debugging, source control, extensions, and an integrated development workflow.

1.1.3 ASP.NET Core

ASP.NET Core was selected for the server-side implementation of Business Buzz.

ASP.NET Core provides the framework through which the system handles server-side application functionality, authentication-related operations, session management, application services, and interaction with the application's data layer.

The framework integrates effectively with C# and the .NET platform. Its dependency-injection functionality also allows application services to be registered and made available to the components that require them.

ASP.NET Core was therefore appropriate for implementing the server-side functionality required by Business Buzz while maintaining separation between the user interface, application logic, and data-access components.

1.1.4 C#

C# was selected as the primary programming language for the server-side implementation of Business Buzz.

C# integrates directly with .NET and ASP.NET Core and supports object-oriented programming principles. This enables the application to be organized into structured and reusable entities, services, interfaces, and other components.

Within Business Buzz, C# is used to implement functionality such as authentication, validation, business-profile processing, personalized journey generation, and database interaction.

Microsoft (n.d.-c) describes C# as a general-purpose programming language within the .NET platform that supports strongly typed and object-oriented application development.

1.1.5 Microsoft SQL Server

Microsoft SQL Server was selected as the relational database management system for Business Buzz.

The database maintains structured information required by the system, including users, business profiles, applications, approval information, funding assessments, journey-related information, and other application records.

A relational database was appropriate because Business Buzz contains multiple related entities and requires relationships and data integrity to be maintained between them.

SQL Server was also suitable because it integrates effectively with the Microsoft and .NET technologies used by Business Buzz and works with Entity Framework Core to support database operations.

Microsoft (n.d.-d) provides SQL Server as a relational database platform for storing, querying, managing, and securing application data.

1.1.6 Entity Framework Core

Entity Framework Core (EF Core**)** was used as the object-relational mapping framework between the Business Buzz C# application and the SQL Server database.

EF Core enables C# entity classes to represent application data and allows database information to be queried and saved through a DbContext. This reduces the need to manually implement every database interaction directly in SQL.

EF Core migrations were also used during development to update the database structure as the Business Buzz data model evolved. For example, changes to the Business Profile entity could be reflected in the SQL Server database through migrations rather than requiring the entire database to be recreated.

Microsoft (n.d.-e) describes Entity Framework Core as an object-relational mapper for .NET.

1.1.7 HTML5

HTML5 was used to define the structure and content of the Business Buzz frontend.

HTML provides the underlying structure for pages presented to users, including authentication pages, the Entrepreneur Dashboard, profile confirmation, registration information pages, and document-related interfaces.

HTML elements were used to organise headings, forms, navigation controls, buttons, links, sections, and other interface components. This provided the foundation upon which CSS styling and JavaScript behaviour could be applied.

MDN Web Docs (n.d.-a) describes HTML as the markup language used to structure web pages and their content.

1.1.8 CSS3

CSS3 was used to implement the visual design and presentation of Business Buzz.

Custom CSS controls the application's colour scheme, typography, spacing, sidebar navigation, cards, buttons, status indicators, dashboards, responsive layouts, and other interface elements.

CSS was particularly important for establishing a consistent Business Buzz visual identity across the application. Reusable CSS classes were used across multiple pages to maintain consistency and reduce unnecessary duplication.

According to MDN Web Docs (n.d.-b), CSS is used to describe the presentation and layout of documents written using markup languages such as HTML.

1.1.9 Bootstrap 5.3

Bootstrap 5.3 was used as a frontend toolkit alongside the custom HTML and CSS developed for Business Buzz. The frontend references Bootstrap 5.3.8 through a Content Delivery Network (CDN).

Bootstrap was selected because it provides responsive frontend utilities and reusable styling functionality. It was used together with custom Business Buzz CSS rather than replacing the application's custom visual design.

This allowed the team to benefit from Bootstrap's responsive functionality while maintaining the specific Business Buzz colour scheme, layouts, buttons, navigation elements, and other interface components.

Bootstrap Team (n.d.) describes Bootstrap as a frontend toolkit designed to support responsive and mobile-first web development.

1.1.10 JavaScript

JavaScript was used to provide client-side behavior and interactivity within the Business Buzz frontend.

JavaScript supports functionality that occurs within the browser, including responding to user-interface events and controlling dynamic frontend behavior.

During frontend development, JavaScript also supported testing of application interactions and page flows while other server-side functionality was still under development.

MDN Web Docs (n.d.-c) describes JavaScript as a programming language used to implement functionality and interactive behavior in web applications.

1.1.11 Git and GitHub

Git and GitHub were used for source-code version control and collaboration between Business Buzz team members.

Git enabled the development team to track changes to the source code and maintain a history of implementation work. GitHub provided the shared remote repository through which team members contributed to the same Business Buzz project.

Separate development branches allowed different components to progress independently without requiring every change to be immediately introduced into the main branch. Changes could be committed and pushed to GitHub and subsequently compared and integrated through the repository workflow.

The Git Project (n.d.) documents Git as a distributed version-control system, while GitHub (n.d.) provides repository-based functionality for storing, managing, and collaborating on source code.

1.2 Extracts of Complex Code

The following extracts of code are the original work of the Business Buzz development team. They were selected because they represent areas of the system where multiple application requirements had to be combined into working functionality.

1.2.1 Authentication and Password Hashing

The authentication service is responsible for registering users and authenticating existing users. This functionality was considered complex because secure authentication requires more than simply comparing a password with a value stored in the database. It also requires passwords to be handled and stored securely.

During registration, the application first checks whether an account already exists for the supplied email address. A new SystemUser is then created, and the supplied password is converted into a secure password hash using the PasswordHasher<TUser> functionality provided by ASP.NET Core Identity before the value is stored.

The Following extract shows the password hashing process:

```
user.PasswordHash = _passwordHasher.HashPassword(
    user,
    password);
_context.SystemUsers.Add(user);
```

Figure 1.1: Password Hashing

During login, the supplied password is verified against the stored password hash rather than comparing or storing the user's original password.

```
var result = _passwordHasher.VerifyHashedPassword( user, user.PasswordHash,password);

if (result == Microsoft.AspNetCore.Identity.PasswordVerificationResult.Failed)
```

Figure 1.2: Password Verification Process

This implementation means that Business Buzz does not need to store users' original passwords. The authentication service also retrieves the user's role during login, allowing the authenticated session to be associated with the appropriate type of system user.

After successful authentication, the application stores the authenticated user's ID and role in the server-side session. This allows subsequent pages to identify the authenticated user rather than relying only on information supplied by the browser. This is consistent with Noni's implemented authentication flow.

1.2.2 Business Profile Creation and Validation

The Business Profile functionality was considered complex because it combines authentication, validation, database verification, and data persistence within one process.

Before a business profile can be created, the application retrieves the authenticated user's ID from the session. This ensures that an entrepreneur cannot create a business profile without first being authenticated.

```
var userId = HttpContext.Session.GetInt32("UserId");

if (userId == null)

{

    return RedirectToPage("/Account/Login");
```

Figure 1.3: User Session Management

The application then validates the information entered by the entrepreneur, including the business name, business type, industry, and business location. The selected business type is also verified against the database before the profile is saved.

```
var userId = HttpContext.Session.GetInt32("UserId");
if (userId == null)
{
    return RedirectToPage("/Account/Login");
```

Figure 1.4: Server-Side Information Verification

Once the information has passed validation, the profile is associated with the authenticated user.

```
var userId = HttpContext.Session.GetInt32("UserId");
if (userId == null)
{
    return RedirectToPage("/Account/Login");
```

Figure 1.5: Linking Onboarding Information to a Business Profile

Associating the profile with the authenticated user allows the Business Profile to become the foundation for later functionality, including the entrepreneur's personalized business journey.

1.2.3 Personalized Journey Generation

The personalized journey functionality was developed so that Business Buzz does not necessarily present every entrepreneur with the same business journey.

The application retrieves the entrepreneur's Business Profile and evaluates information contained within it. The journey service then determines which journey stages are applicable to that particular business.

For example, the EmploysPeople property can determine whether a UIF-related stage should be included, while the SeeksFunding property can determine whether the Funding Readiness stage should be presented to the entrepreneur.

```
var userId = HttpContext.Session.GetInt32("UserId");
if (userId == null)
{
    return RedirectToPage("/Account/Login");
```

Figure 1.6: Business Journey Personalization

The journey rules are maintained within the application's infrastructure service rather than being placed directly in the user interface. This separation of responsibilities makes the application easier to maintain and allows the business journey to adapt according to information supplied by the entrepreneur rather than displaying a fixed sequence of stages to every user.

1.3 Source Code References

External documentation and framework functionality were consulted during the development of Business Buzz. The application-specific code was developed for the Business Buzz project; however, official framework documentation was used to understand and correctly implement functionality provided by the frameworks and libraries used by the application.

1.3.1 Password Hashing and Verification

Description

Business Buzz uses the Password Hasher<TUser> class provided by ASP.NET Core Identity for password hashing and verification.

The development team did not create a custom password-hashing algorithm. Instead, the Business Buzz authentication service uses framework-provided functionality to generate a hashed representation of a user's password during registration and verify a supplied password during login.

Microsoft documents Password Hasher<TUser> as providing Identity password-hashing functionality, including the HashPassword and VerifyHashedPassword methods.

Usage

The functionality is used within:

- AuthenticationService.RegisterAsync()
- AuthenticationService.LoginAsync()
- User registration functionality

- User login functionality

Microsoft. (n.d.-f). *PasswordHasher<TUser> class*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.passwordhasher-1

1.3.2 ASP.NET Core Session State

Description

ASP.NET Core session functionality was used to maintain information about an authenticated user between requests.

After successful login, Business Buzz stores the authenticated user's ID and role in the server-side session. Subsequent functionality, such as Business Profile creation, retrieves the user ID from HttpContext.Session to determine which authenticated user is performing the operation.

Microsoft's ASP.NET Core documentation describes session state as a mechanism for maintaining user-specific data while a user browses an application.

Usage

Session functionality is used within:

- Program.cs
- Login functionality
- CreateProfile.cshtml.cs
- Dashboard and user-specific functionality

1.3.3 Entity Framework Core Database Migrations

Description

Entity Framework Core migrations were used to keep the SQL Server database structure synchronized with changes made to the Business Buzz entity model.

For example, when the SeeksFunding property was added to the BusinessProfile entity, a new migration was created and applied to the existing database rather than recreating the database.

EF Core migrations allow a database schema to be incrementally updated as the application's data model changes while preserving existing data.

Usage

Migrations are used within:

- BusinessBuzz.Infrastructure
- EF Core migration files

- Business Profile entity changes
- SQL Server database updates

1.3.4 Entity Framework Core Object-Relational Mapping

Description

Entity Framework Core was used to allow Business Buzz C# entity classes to interact with the relational SQL Server database.

The framework provides object-relational mapping functionality, allowing database data to be queried and saved through C# objects and a DbContext. This functionality provides the basis for database operations throughout the application.

Noni's implementation identifies EF Core as being used for retrieving users, business profiles and journey stages and for saving new business profiles.

Usage

Entity Framework Core is used within:

- BusinessBuzzDbContext
- Authentication Service
- Business Profile functionality
- Journey Service
- Business Buzz entity classes

Microsoft. (n.d.-e). *Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/

1.3.5 Bootstrap 5.3 Frontend Toolkit

Description

Bootstrap was incorporated into the Business Buzz frontend through its CDN stylesheet to provide responsive frontend utilities and baseline styling functionality.

Bootstrap is used together with custom HTML and CSS developed specifically for Business Buzz. Custom CSS controls the application's project-specific colour scheme, layouts, navigation components, cards, buttons, and other visual elements.

Usage

Bootstrap is referenced within frontend interfaces, including:

- Authentication pages
- Entrepreneur Dashboard
- CIPC Registration page
- Digital Documents page
- Other Entrepreneur portal interfaces

Bootstrap Team. (n.d.). *Get started with Bootstrap*. Bootstrap. https://getbootstrap.com/docs/5.3/getting-started/introduction/

1.4 Problems Encountered

The following problems were encountered during the implementation of Business Buzz. This section reflects on the development challenges experienced by the team and explains the solutions applied to resolve them.

1.4.1 Entity Framework Core Migration and Database Configuration

One of the problems encountered during development was ensuring that the Entity Framework Core model could be successfully converted into a SQL Server database.

The project contains multiple related entities, including users, business profiles, applications, approval reviews, and funding assessments. Some of these relationships created multiple possible cascade-delete paths in SQL Server.

This resulted in a database-creation error because SQL Server does not permit certain multiple cascade paths.

The issue was resolved by explicitly configuring the delete behavior of selected relationships rather than relying entirely on the default cascade-delete behavior. After the relationships were configured appropriately, the database migration completed successfully.

1.4.2 Authentication Service Registration

Another problem occurred when the application attempted to access the registration functionality but could not resolve IAuthenticationService.

The problem was caused by the authentication service not being correctly available through the application's dependency-injection configuration.

The service was subsequently registered in Program.cs using the following configuration:

```
Builder.Services.AddScoped<
IAUthenticationService,
Authentication Service>();
```

After correcting the dependency-injection configuration and ensuring that the correct project version was being executed, the registration functionality operated successfully.

1.4.3 SQL Server Connection Timeout During Login

During testing of the login functionality, the authentication code successfully reached the database query, but SQL Server did not consistently complete the connection during the post-login process.

The resulting error was a database connection timeout rather than an error in the password-verification logic.

The SQL Server instance was restarted and the database connection was tested again. After the SQL Server service was restored, the login query executed successfully and authentication testing could continue.

1.4.4 Updating the Database as the Application Evolved

As new functionality was introduced into Business Buzz, changes to the entity model also had to be reflected in the database.

For example, implementation of the personalized journey required the Business Profile to record whether an entrepreneur was seeking funding. A new Seeks Funding property was therefore added to the Business Profile entity.

An Entity Framework Core migration was subsequently created and applied to SQL Server. This enabled the database structure to evolve with the application without requiring the existing database to be recreated and allowed development to continue while preserving existing data.

1.4.5 Configuring the ASP.NET Core Frontend Project

During the initial implementation of the Business Buzz frontend, difficulty was experienced when running the application correctly from Visual Studio 2022.

Visual Studio initially opened the project in Folder View, and the run configuration displayed Current Document (Program.cs) rather than running the Business Buzz frontend project itself. As a result, the application was not initially being started using the expected project configuration.

The problem was resolved by opening the BusinessBuzz.Frontend.sln solution file in Visual Studio and ensuring that the Business Buzz frontend project was configured as the startup project.

After correcting the configuration, the application could be executed normally using the project's launch settings.

1.4.6 Static Files and Web Root Path

Another frontend problem occurred when the ASP.NET Core application attempted to locate the wwwroot directory from an incorrect location.

During execution, the application produced a warning indicating that the web root path could not be found under the bin\Debug\net8.0\wwwroot directory. This affected the expected handling of static frontend resources because the Business Buzz HTML, CSS, and JavaScript files were located within the actual project's wwwroot directory.

The issue was related to the manner in which the application had initially been launched rather than the wwwroot directory being absent.

After opening the correct solution and running the frontend as the ASP.NET Core project, the correct content root was used and the application served the required static files.

1.4.7 Navigation Between Frontend Pages

During development of the Entrepreneur portal, some navigation controls appeared correctly on the interface but did not navigate to the required pages when selected.

For example, the Continue control on the Entrepreneur Dashboard was intended to navigate to the CIPC Registration page. It was originally implemented as an HTML <button> without page-navigation behavior attached to it. Consequently, selecting the control did not open the required page.

The problem was resolved by replacing controls intended specifically for page-to-page navigation with HTML anchor (<a>) elements containing the appropriate href destination.

The same approach was subsequently applied to the **Generate DigiDoc** control. Custom CSS classes were applied to these anchor elements so that they retained the appearance of Business Buzz buttons while providing reliable navigation.

1.4.8 Developing the Frontend and Backend in Parallel

The frontend and backend components of Business Buzz were developed by different team members in parallel. This created a challenge because several frontend screens needed to be implemented and tested before all corresponding backend functionality was available.

Waiting for the entire backend to be completed before implementing the frontend would have delayed development.

To address this problem, temporary sample data was used to represent information that would eventually be obtained from the backend. Temporary frontend navigation and behavior were also implemented so that the application's interface and user journey could be tested.

This approach allowed the page structure, styling, and navigation flow to be developed independently while backend development continued. The temporary data and behavior can subsequently be replaced with integrated server-side functionality as the components are combined.

1.4.9 Collaborating Through Git and GitHub

Business Buzz contains frontend, backend, and infrastructure components developed by different team members. Managing changes within a single repository created a risk that one developer's work could conflict with changes made by another developer.

Git and GitHub were used to manage this challenge.

Development work was performed using separate branches, allowing changes to be committed and pushed independently. Frontend development, for example, could be maintained on its own development branch rather than every change being made directly to the main branch.

Pull requests could subsequently be used to compare development branches with the main branch before integration. This provided a controlled approach to combining the different components of Business Buzz while maintaining a history of the team's development work.

References

Bootstrap Team. (n.d.). *Get started with Bootstrap*. Bootstrap. https://getbootstrap.com/docs/5.3/getting-started/introduction/

Git Project. (n.d.). *Git documentation*. https://git-scm.com/docs/git

GitHub. (n.d.). *Repositories documentation*. GitHub Docs. https://docs.github.com/en/repositories

MDN Web Docs. (n.d.-a). *HTML: HyperText Markup Language*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTML

MDN Web Docs. (n.d.-b). *CSS: Cascading Style Sheets*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/CSS

MDN Web Docs. (n.d.-c). *JavaScript guide*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Microsoft. (n.d.-a). *What is Visual Studio?* Microsoft Learn. https://learn.microsoft.com/en-us/visualstudio/get-started/visual-studio-ide

Microsoft. (n.d.-b). *Visual Studio Code documentation*. Visual Studio Code. https://code.visualstudio.com/docs

Microsoft. (n.d.-c). *C# documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/

Microsoft. (n.d.-d). *SQL Server technical documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/sql/sql-server/

Microsoft. (n.d.-e). *Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/

Microsoft. (n.d.-f). *PasswordHasher<TUser> class*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity.passwordhasher-1

Microsoft. (n.d.-g). *Session and state management in ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state

Microsoft. (n.d.-h). *Migrations overview*. Microsoft Learn. https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/